

Mechanizing Nested Hybrid Arguments

Markus Krabbe Larsen
Theoretical Computer Science
IT University of Copenhagen
Copenhagen, Denmark
krml@itu.dk

Carsten Schürmann
Theoretical Computer Science
IT University of Copenhagen
Copenhagen, Denmark
carsten@itu.dk

Abstract—Hybrid arguments are prevalent in cryptographic security proofs, however, existing mechanizations in proof assistants are not as general as they could be. In this paper, we prove a general theorem about the admissibility of hybrid arguments that encompasses the query-counting, multi-instance, and nested case. We present three case studies about public-key encryption, which demonstrate the generality of the theorem. The results of this paper are achieved in the setting of state-separating proofs and the theory is integrated into and the case studies are mechanized in Nominal-SSProve.

I. INTRODUCTION

Hybrid arguments [23], [15] are a well-known technique to reason about the correctness and security of cryptographic protocols, especially in the setting of code-based game-playing proofs [7]. In a hybrid argument, two games are shown to be indistinguishable by constructing a series of *hybrids* whose consecutive indistinguishability is easier to reason about.

Hybrid arguments have been formalized before [1], but previous formalizations are limited to what we call *query-counting* hybrid arguments. In a query-counting hybrid argument, the behavior of the hybrids is exclusively determined by the number queries to a specific oracle i.e. the i 'th hybrid answers the first i queries by one method and the rest of the queries are answered by another method.

Another form of hybrid arguments is about the security of *multi-instance* cryptographic protocols, that considers an arbitrary but fixed number of instances of the protocol running in parallel [5]. The idea is similar to reasoning about the security of a protocol in the symbolic model, where we assume that there are arbitrary but finitely many instances of the protocol running in parallel, and the attacker has access to the different runs. The Woo-Lam protocol [22], for example, is secure in the single-instance but insecure in the multi-instance setting. This form of hybrid argument cannot be captured with a query-counting hybrid argument alone, because the order in which the adversary interacts with the instances can be interleaved.

In addition, we wish to support *nested* hybrid arguments. An example illustrating the usefulness of nested hybrid arguments is to prove the multi-instance, many-time security under a chosen plaintext attack (CPA) of ElGamal encryption. The proof is well known and proceeds by two nested hybrid arguments. The outer argument uses a *multi-instance* hybrid argument to reason about multiple instances of the protocol,

and the inner argument uses a *many-time* hybrid argument [5]. A multi-instance reduction can be thought of as composing the single-instance security proofs into one big composed security proof. The induced notion of compositionality resembles that defined in the *universal compositionality* (UC) framework, and we believe that this work could be considered a first step towards mechanizing cryptographic arguments in UC. We leave it to future work to explore this observation and to relate it to other work on UC in EasyCrypt, for example EasyUC [10].

The methodology of this paper is to encode cryptographic protocols as state-separating proofs [9] (SSPs). This abstraction gives us enough flexibility to state and mechanize one general hybrid argument theorem (Theorem 4) that models both query-counting and multi-instance arguments. Hybrid arguments may be nested. As a case study, we mechanize the proof that ElGamal is many-time multi-instance CPA-secure by reducing it to the decisional Diffie-Hellman assumption solely from the fact that ElGamal is one-time CPA-secure. We validate our findings by encoding all claims presented in this paper in Nominal-SSProve [19] where the nominal features of Nominal-SSProve (now fully integrated into SSProve [17]) are critical. This is to our knowledge the first uniform treatment of nestable many-time and multi-instance hybrid arguments.

In state-separating proofs security properties are modeled as game pairs, where we distinguish the *real* game describing a concrete implementation of a cryptographic algorithm from the *ideal* game, which describes – operationally – its specification. Together, the two games mathematically define the security property that stipulates the existence of a bound on the advantage of a (computationally bounded) adversary to distinguish between the real and the ideal game.

In Nominal-SSProve, SSPs are modeled using an embedded system of packages inheriting strong reasoning principles from the host system Rocq. State is local to each package and basic properties such as disjointness of state is informally assumed. Nominal-SSProve guarantees disjointness properties by using nominal sets. Besides cryptographic games, adversaries are also modeled as packages in Nominal-SSProve. In particular, sequential package composition allows us to express that the adversary $\mathcal{A}$ interacts with game G , written as $\mathcal{A} \circ G$, where implementations in G are substituted for calls in $\mathcal{A}$.

To prove the indistinguishability (up to ϵ) of G^0 and G^1 means that we need to show that $\mathcal{A}$ has only an ϵ -advantage in distinguishing G^0 from G^1 , written as $\text{Adv}_{G^0, G^1}(\mathcal{A}) \leq \epsilon$

and defined as

$$\text{Adv}_{G^0, G^1}(\mathcal{A}) = \left| \Pr[\text{true} \leftarrow (\mathcal{A} \circ G^0).\text{RUN}()] - \Pr[\text{true} \leftarrow (\mathcal{A} \circ G^1).\text{RUN}()] \right|, \quad (1)$$

where $\Pr[\text{true} \leftarrow P.\text{RUN}()]$ is the probability that evaluating function RUN in $P = \mathcal{A} \circ G^0$ or $P = \mathcal{A} \circ G^1$ returns true . As mentioned above, the operator $\circ$ stands for sequential package composition, where each call in $\mathcal{A}$ is either resolved to a function declared in G^0 or G^1 , depending on which package $\mathcal{A}$ is composed with.

We turn now to the idea behind hybrid arguments. Let M be a game-pair describing the security property to be proven and G a game-pair assumed to be secure. A hybrid argument is a technique that reduces the indistinguishability of M^0 and M^1 to the indistinguishability of G^0 and G^1 . It does so by a series of $n + 1$ hybrids $(H_0, \dots, H_n)$, where the indistinguishability of each pair of consecutive hybrids reduces to the indistinguishability of G^0 and G^1 . Here M^0 is indistinguishable from $H_0 \circ G^0$ and $H_n \circ G^0$ is indistinguishable from M^1 . These assumptions are formalized as follows.

$$M^0 \approx_0 H_0 \circ G^0 \quad (2)$$

$$H_i \circ G^1 \approx_0 H_{i+1} \circ G^0 \text{ for all } 0 \leq i < n \quad (3)$$

$$H_n \circ G^0 \approx_0 M^1 \quad (4)$$

Then we can show that for all $\mathcal{A}$ we have

$$\text{Adv}_M(\mathcal{A}) = n \cdot \text{Adv}_G(\mathcal{A} \circ H_\S)$$

where $H_\S$ (internally) chooses its index uniformly at random.

A mechanized proof of the hybrid argument Theorem 4 is the main contribution of our paper. To our knowledge this is the first reusable and nestable treatment of hybrid arguments. It is also the first reusable hybrid argument for multi-instance reductions.

As an example for how to use hybrid arguments in Nominal-SSProve we introduce a public-key encryption (PKE) primitive and show the following three case studies. The names of the game-pairs formally introduced in Section V are given in parentheses.

- 1) We show by a query-counting hybrid argument that for all implementations of a PKE primitive, many-time indistinguishability under CPA-security (MT_CPA) can be reduced to one-time indistinguishability under CPA-security (OT_CPA).
- 2) We show by a hybrid argument that for all implementations of a PKE primitive, multi-instance many-time CPA-security (MI_MT_CPA) reduces to single-instance many-time CPA-security (MT_CPA).
- 3) We show by nesting the hybrid arguments of 1) and 2) that the multi-instance many-time security of ElGamal (MI_MT_CPA instantiated) reduces to the decisional Diffie-Hellman assumption (DDH).

We remark that our techniques apply also to stronger notions of security, such as security under an adaptive chosen ciphertext attack (CCA2). The only change in the mechanization is to add additional oracles. See [9] for a definition and see [17]

for a mechanization of CCA2-security for PKE as part of the KEM-DEM development in the context of state-separating proofs.

A. Contributions

- We prove a general theorem about the admissibility of hybrid arguments.
- We mechanize the proof and integrate it into Nominal-SSProve.
- We provide a robust definition for indistinguishability under chosen plaintext attack (CPA) security for public-key encryption and mechanize it in Nominal-SSProve.
- We give a complete proof of the reduction from many-time, multi-instance CPA-security to one-time CPA-security.

B. Related Work

Hybrid arguments have been studied in other proof assistants. EasyCrypt, for example, supports reasoning about cryptographic protocols in the computational model. Query-counting hybrid arguments are introduced in form of a reasoning principle in EasyCrypt in [1], where they are used to verify secure function evaluation using garbled circuits. State-separating proofs are encodeable in EasyCrypt at a significant cost, and the high-level algebraic reasoning is not attainable [13], [12]. This is in contrast to this work, where we prove the admissibility of hybrid arguments using high-level algebraic reasoning about packages as state-separating proofs.

CPA-security for public-key encryption schemes has been studied extensively [6]. The fact that one-time CPA-security (OT_CPA) implies many-time CPA-security (MT_CPA) is “textbook-material” and an SSP-style proof is given in [21, Chapter 14].

Previous mechanizations of CPA-security for a public-key encryption scheme usually consider only OT_CPA for a specific implementation such as ElGamal. This is the case for the developments in CertiCrypt [4], EasyCrypt [3], [18] and FCF [20]. The source code of EasyCrypt¹ contains a mechanized proof that OT_CPA implies MT_CPA, but it does not seem to be part of any published work. One of the examples given in ProofFrog [14] is the reduction from MT_CPA to OT_CPA by a hybrid argument.

Earlier versions of SSProve [17]² provided definitions for OT_CPA- and MT_CPA-security, but as discovered in [19] the definitions were flawed in their given form: every time the adversary queries the encryption oracle (called QUERY in this work) a new set of keys is generated. As a consequence it is not possible to construct a reduction from OT_CPA to MT_CPA, since it crucially relies on the public key being fixed throughout the game. However the mechanization that we present in this paper for Nominal-SSProve is correct: We provide robust and re-usable definitions for OT_CPA and

¹https://github.com/EasyCrypt/easycrypt/blob/b3f6888/theories/encryption/PKE_hybrid.ec

²<https://github.com/SSProve/ssprove/blob/b5b89d6/theories/Crypt/examples/AsymScheme.v>

MT_CPA as state-separating style games and mechanize the parametric reduction from OT_CPA to MT_CPA.

Complex hybrid arguments are mechanized in [8], [2]. They are all instances of the query-counting hybrid argument and the nesting of hybrid arguments is by custom reasoning rather than by generic theorem.

C. Organization

This paper is organized as follows. In Section II we introduce Nominal-SSProve. Next, in Section III, we formalize the law of total probability. In Section IV, we discuss hybrid arguments and prove the hybrid argument theorem (Theorem 4). We describe the mechanization of hybrid arguments in Nominal-SSProve. In the following Section V, we turn to public-key encryption (PKE) schemes and show how to reduce many-time CPA-security to one-time CPA-security for any PKE scheme and multi-instance to single-instance CPA-security, respectively. Finally, we construct a formal proof of multi-instance many-time CPA-security for ElGamal. We conclude in Section VI and assess results. A description of the artifacts³ attached to this submission is given in Appendix A.

II. NOMINAL-SSPROVE

In the computational model, security properties about cryptographic protocols are typically expressed as game-pairs, representing the *real* and the *ideal* functionality. The typical act of proving a protocol correct is tantamount to reducing the advantage of an attacker to distinguish between the two games to the advantage of solving a hard problem, such as the decisional Diffie-Hellman problem. The overall structure of such a *game-hopping* argument is as follows. We compute the bound for the attacker to distinguish the game-pair defining the security property to be proven by reducing it – in several steps – to the advantage of distinguishing game-pairs that represent the computational assumptions. The overall bound is then computed by combining the bounds derived for the respective steps using the triangle inequality, computational assumptions, and other mathematical facts.

Nominal-SSProve implements state-separating proofs (SSPs) [9]. SSPs defines a package system for probabilistic programs with suitable notions of interface, package composition, package equivalence, and package rewriting. The organizational force of SSPs is ultimately responsible for the main results of this work. The results could carry over to other systems such EasyCrypt, if EasyCrypt were to support SSPs in a natural way.

Cryptographic primitives are modeled as record types that declare the names of the base types and the types of the cryptographic algorithms. Sample primitives include public-key encryption, commitment schemes, symmetric encryption, Σ -protocols, and so on.

Implementations of a primitive are represented as values of the corresponding record type. Such a value is an n -tuple that declares base types, and gives concrete implementations of

the required cryptographic algorithms (as functions). Examples of implementations include ElGamal, Paillier, Schnorr (for Σ -protocols), Pedersen commitments, and so on.

Security properties are modeled as packages representing game-pairs. Each package is indexed by a bit encoding the real or ideal game. Oracle calls are modeled as function calls into other packages. Interfaces model which functions are exported/imported from this package. A security property is always parameterized by implementations stating which cryptographic primitives are provided. Examples of security properties are CPA-security, special honest verifier zero-knowledge (for Σ -protocols), hiding (for commitment schemes), and so on.

Advantage bounds represent an upper bound of the advantage that an adversary has to distinguish successfully between the two games contained within a game-pair. Examples of security bounds are that CPA-security for ElGamal is bounded by the advantage to distinguish the DDH game-pair, and special honest verifier zero-knowledge for Schnorr is bounded by 0.

A. Language

Nominal-SSProve provides a stateful probabilistic language for writing security games-as-packages and is embedded into the Rocq framework via a free monad called `code`. The free monad is defined in continuation-passing-style and provides the primitive and derived actions depicted in Figure 1, encompassing probabilistic sampling, and assertions. Although Nominal-SSProve provides more language actions than those depicted in the figure, this is a complete set of actions that are used in code listing in this paper.

Nominal-SSProve defines a submonad of `code` called `dist`, which is restricted to actions `sample` and `ret` and represents the fragment of stateless probabilistic algorithms as they are typically used for defining cryptographic primitives.

`code` is embedded into Rocq, which means that expressions of type `code` using continuation `k` may also be constructed using standard Rocq constructs, such as conditionals `if .. then .. else` and `let` bindings `let .. := .. in ...`. All syntactic constructs support pattern-matching when prefixing a binder with an apostrophe `'`.

B. Packages

A package is a collection of functions that implements the functionality related to the task at hand: a package can implement an adversary, a security game, or a reduction. A package is defined by input and output interfaces, it must define all functions according to the output interface and call any functions according to the input interface. Locations are local to a package and preserved between function calls. Locations store values such as public keys, nonces, message history, counters, and the like.

Types, signatures, interfaces, declarations, and packages are given by the following grammar.

³<https://github.com/MarkusKL/ssprove/tree/for-submission-jan26>

Fig. 1. Nominal-SSProve language features (selected)

<code>x ← sample D ;; k x</code>	Sample from the given distribution <code>D</code>
<code>#put l := v ;; k</code>	Store value <code>v</code> at location <code>l</code>
<code>x ← get l ;; k x</code>	Read from location <code>l</code>
<code>x ← call [F] v ;; k x</code>	Call function <code>F</code> with argument <code>v</code>
<code>ret v</code>	Monadic return of value <code>v</code>
<code>x ← k1 ;; k2 x</code>	Use the result of <code>k1</code> in <code>k2</code> (monadic bind)
<code>#assert v ;; k</code>	Check that <code>v</code> is <code>true</code> , otherwise the program aborts
<code>x ← getSome l ;; k x</code>	Get the value of location <code>l</code> and unwrap the <code>Some</code> of an <code>option</code> value, otherwise the program stops
<code>x ← get l [i] ;; k x</code>	Get the value of location <code>l</code> at array index <code>i</code>
<code>#put l [i] := v ;; k</code>	Put the value <code>v</code> to location <code>l</code> at array index <code>i</code>

$T ::= \text{unit} \mid \text{bool} \mid \text{'fin } n \mid T_1 \times T_2 \mid \dots$
 $S ::= [F] : \{ T_1 \rightsquigarrow T_2 \}$
 $I ::= [\text{interface } S_1 ; S_2 ; \dots ; S_n]$
 $D ::= [F] \times \{ c \}$
 $P ::= [\text{package } L ; D_1 ; D_2 ; \dots ; D_n]$

T shows the syntax of the relevant types for values passed between packages. `unit` is the singleton type with value `tt`. `'fin n` is the finite type of size `n`. $T_1 \times T_2$ is the product type of T_1 and T_2 . In the code listings, T is referred to as *choice_type*. Interfaces I are defined by signatures $S_1, \dots, S_n$ over identifiers $F_1, \dots, F_n$. P denotes the syntax of packages defined by locations L and declarations $D_1, \dots, D_n$ over identifiers $F_1, \dots, F_n$.

These constructs are sufficient to model games-as-packages, reductions-as-packages, and adversaries-as-packages. In SSP, packages with no imports are called *games*. In contrast, packages with imports are used to model *reductions*. An adversary is modeled as a package that exports one function called `RUN` (see (Equation 1)) and each import corresponds to an oracle that the adversary might call. Note that counting the number of times an adversary has accessed an oracle is not provided by SSP, but needs to be implemented explicitly on a case by case basis.

C. Composition

Adversarial advantage, i.e. the probability of the adversary to guess correctly which game he is interacting with, is based on interaction, which is expressed formally using package composition. In SSP, packages P and P' can be composed sequentially written $P \circ P'$, which corresponds to inlining function calls in the left package by implementations from the right package – or they can be composed in parallel written as $\frac{P}{P'}$, which corresponds to creating a new package resulting from merging packages P and P' . Note, that this operation requires the function names declared in P and P' to be disjoint. When combining packages state-variables are tacitly renamed to avoid variable capture [19].

Two packages that are equal up to variable renaming are said to be α -equivalent, written as $P \equiv P'$. Sequential composition is associative up to α -equivalence i.e.

$$(P \circ P') \circ P'' \equiv P \circ (P' \circ P'').$$

Additionally, sequential composition has left and right identities such that a package P with imports I and exports E satisfies

$$\text{ID}(E) \circ P \equiv P \quad P \circ \text{ID}(I) \equiv P.$$

The identity package $\text{ID}(I)$ for an interface I defines a package that has both imports I and exports I . It defines functions according to I by calls to the underlying interface.

The empty package is given by the identity package over the empty interface $\text{ID}(\{\})$. It is the left and right identity of parallel composition.

$$\frac{\text{ID}(\{\})}{P} \equiv P \quad \frac{P}{\text{ID}(\{\})} \equiv P$$

The interaction between sequential and parallel composition is captured by the interchange rule, which states that

$$\frac{P_1}{P_3} \circ \frac{P_2}{P_4} \equiv \frac{P_1 \circ P_2}{P_3 \circ P_4}$$

when the exports of P_2 and P_4 are disjoint.

Finally, parallel composition is associative and commutative, but we do not rely on these facts in the technical development below.

D. Advantage

Let I be an interface. An I -game is a game with no imports and exports I . In contrast, an I -adversary has imports I and exports one function `[ RUN ] : unit  $\rightsquigarrow$  bool`. Recall from Equation (1) that the advantage between two I -games G and G' for a specific I -adversary $\mathcal{A}$ is defined as

$$\begin{aligned} \text{Adv}_{G,G'}(\mathcal{A}) = & |\Pr[\text{true} \leftarrow (\mathcal{A} \circ G).\text{RUN}()] \\ & - \Pr[\text{true} \leftarrow (\mathcal{A} \circ G').\text{RUN}()]|, \end{aligned}$$

where $\Pr[\text{true} \leftarrow P.\text{RUN}()]$ is the probability that evaluating function `RUN` in $P = \mathcal{A} \circ G$ or $P = \mathcal{A} \circ G'$ returns `true`. Directly from this definition follows the triangle inequality

$$\text{Adv}_{G,G''}(\mathcal{A}) \leq \text{Adv}_{G,G'}(\mathcal{A}) + \text{Adv}_{G',G''}(\mathcal{A})$$

for I -games G, G', G'' , and I -adversary $\mathcal{A}$, which allows us to “hop” through an intermediate game. Furthermore, advantage is symmetric and α -equivalent games have advantage 0, i.e.

$$\text{Adv}_{G,G'}(\mathcal{A}) = \text{Adv}_{G',G}(\mathcal{A}) \quad G \equiv G' \Rightarrow \text{Adv}_{G,G'}(\mathcal{A}) = 0$$

for I -games G , G' and I -adversary $\mathcal{A}$. From associativity of sequential composition we derive the reduction lemma.

$$\text{Adv}_{R \circ G, R \circ G'}(\mathcal{A}) = \text{Adv}_{G, G'}(\mathcal{A} \circ R)$$

for I -games G , G' , an E -adversary $\mathcal{A}$ and a reduction expressed as package R with imports I and exports E .

E. Perfect indistinguishability

In the case that the advantage is 0 we say that I -games G and G' are *perfectly indistinguishable*, which means that $\text{Adv}_{G, G'}(\mathcal{A}) = 0$, written as $G \approx_0 G'$. For this to be a meaningful definition it is essential that the games and the adversary do not share state-variables, which is assumed for SSP and automatically provided by the nominal sets constructions in Nominal-SSProve.

The fact that two packages are perfectly indistinguishable is shown in the probabilistic relation Hoare logic (pRHL). Due to its nominal features, in Nominal-SSProve, when reasoning about advantages, games may be substituted for one another as long as they are perfectly indistinguishable.

Like prior formalizations, Nominal-SSProve does not provide an elegant mechanism to define the notion of *probabilistic polynomial-time* (PPT) adversary. The main challenge is, as usual, that adversaries are modeled as functions natively supported by the respective type theory, and those functions are at the very least in EXPTIME. Thus, it remains a task for a human verifier to admit that the algorithms run in PPT. This also means that we omit the security parameter from our mechanization.

Table 2 shows the translation of concepts presented as SSP into the syntax of Nominal-SSProve as used for formulating theorems throughout this paper, and the proofs in the supplementary materials. For example, we shall use type package $I \ E$ to denote a package with imports I and exports E and a predicate $\text{Package } I \ E \ P$ that shows that a package P has imports I and exports E . A similar comment holds for adversaries and games. The other translations are straightforward.

III. THE LAW OF TOTAL PROBABILITY

As a first step towards hybrid arguments, we present the law of total probability, which is necessary to prove a uniform bound for the advantage of the adversary in the hybrid argument. Mathematically, the law of total probability is stated as follows: For a probability event $A \subseteq \Omega$ and a mutually exclusive family of events B_x covering Ω for $x \in U$ we have

$$\Pr[A] = \sum_{x \in U} \Pr[A \mid B_x] \Pr[B_x]. \quad (5)$$

The idea of “covering Ω ” is equivalent to the notion of losslessness, which requires that

$$\sum_{x \in U} \Pr[B_x] = 1.$$

We need to refine the formulation of Equation (5) to adjust it to the setting of SSPs. Consider the case where $A = v \leftarrow c$ for a value $v \in V$ and a distribution c , while $B_x = (v = x)$

Fig. 2. Correspondence between SSP concepts and Nominal-SSProve

State-separating proofs	Notation	Nominal-SSProve
Interface	I, E	$I \ E : \text{Interface}$
Package	P	$P : \text{package } I \ E$
- as predicate		$\text{Package } I \ E \ P$
I-adversary	$\mathcal{A}$	$A : \text{adversary } I$
- as predicate		$\text{Adversary } I \ A$
I-game	G	$G : \text{game } I$
- as predicate		$\text{Game } I \ G$
Linking	$P \circ P'$	$P \circ P'$
Parallel	$\frac{P}{P'}$	$P \parallel P'$
Interface union	$I \cup I'$	$\text{union } I \ I'$
No locations	$\{\}$	empty
List of locations	$\{l_1, \dots, l_n\}$	$[\text{fmap } l1; \dots, l_n]$
Advantage	$\text{Adv}_{G, G'}(\mathcal{A})$ $\text{Adv}_{G^0, G^1}(\mathcal{A})$ $G \approx_0 G'$	$\text{Adv } G \ G' \ A$ $\text{AdvOf } G \ A$ $\text{perfect } I \ G \ G'$
Perfect indistinguishability		

Fig. 3. The IPICK interface and the RAND and CONST packages.

```

Definition IPICK T :=
  [interface [ PICK ] : { unit -> T }].

Definition RAND {T : choice_type} (c : dist T)
  : game (IPICK T) :=
  [package [fmap cell T] ;
  [ PICK ] (-) {
    mr ← get cell T ;;
    if mr is Some r then
      ret r
    else
      r ← c ;;
      #put cell T := Some r ;;
      ret r
  } ].

Definition CONST {T : choice_type} (x : T)
  : game (IPICK T) :=
  [package empty ;
  [ PICK ] (-) {
    ret x
  } ].

```

and $U = V$ i.e. A represents the event that the distribution c has value v and B_x represents the event that v is equal to x .

$$\Pr[v \leftarrow c] = \sum_{x \in V} \Pr[v \leftarrow c \mid v = x] \Pr[v = x] \quad (6)$$

Intuitively, we can think of the sum as a case analysis on the result v of sampling from distribution c weighted by the corresponding probability of picking exactly v . As long as V contains all values possibly produced by c , this choice of B_x is lossless.

However, there is still a discrepancy: the probability events that we are working with are of the form $\Pr[\text{true} \leftarrow (\mathcal{A} \circ G).\text{RUN}()]$, so we construct packages that correspond to Equation 6, but that also take the adversary's interaction into account. To this extent, we introduce two packages, namely **RAND** and **PICK** depicted in Figure 3. The **RAND** package is parameterized by a distribution c and serves the role of $\Pr[v \leftarrow c]$ by picking a value according to the distribution c at most once. The **CONST** package is parameterized by a value x and serves the role of $\Pr[v = x]$ by always picking the value x . Both packages implement the **IPICK** interface. This leads us to the following theorem.

Theorem 1 (The Law of Total Probability). *For all IPICK-adversaries $\mathcal{A}$ and lossless distributions c over V ,*

$$\begin{aligned} & \Pr[\text{true} \leftarrow (\mathcal{A} \circ \text{RAND } c).\text{RUN}()] \\ &= \sum_{x \in V} \Pr[x \leftarrow c] \Pr[\text{true} \leftarrow (\mathcal{A} \circ \text{CONST } x).\text{RUN}()]. \end{aligned}$$

Proof. We proceed by induction over the structure of the **RUN** function of the adversary $\mathcal{A}$. The only interesting inductive case is that of **call**. On the adversary's first invocation to **PICK** the behavior of **RAND** and **CONST** are different. However, we can align the sampling performed by **RAND** on the left-hand side with the weighted sum on the right-hand side. It remains to show that for each possible picked value x , further invocations of **PICK** produce the same result. Indeed it does, because **RAND** will keep returning the now fixed value x , which matches the behavior of **CONST**. $\square$

The theorem is displayed as Rocq code as follows:

```
Theorem TotalProbability {T : choice_type}
  (c : dist T) {LosslessCode T c}
  {A} {Adversary (IPICK T) A} :
  Pr' (A ∘ RAND c) true =
    (\dlet_ (x <- Pr_fst c)
      Pr' (A ∘ CONST x)) true.
```

The notation $\backslash\text{dlet}_-(x \leftarrow \text{Pr_fst } c)$ represents the weighted sum over distribution c . Note that c is required to be lossless, so that, in the case that an adversary chooses not to interact with the game, i.e. never calls **PICK**, only the right-hand side of the equation samples from c . Because of losslessness, we can disregard that we ever did the sampling.

IV. HYBRID ARGUMENTS

Next we present the hybrid argument theorem. Recall from the introduction that hybrid arguments are about two games M^0 and M^1 to be shown indistinguishable up to ϵ keeping in mind that this game pair describes a security property, such as many-time CPA-security or similar. Our aim is to construct a hybrid argument to support reasoning about the indistinguishability of such a game-pair, which we describe and prove admissible in Theorem 4 below. But first, we prove a lemma illustrating a useful package identity that we use in the proof.

Lemma 2. *For interface E_1 , interface E_2 , E_1 -game G_1 , and E_2 -game G_2 it holds that*

$$\frac{G_1}{\text{ID}(E_2)} \circ G_2 \equiv \frac{\text{ID}(E_1)}{G_2} \circ G_1$$

Proof. Remember that the import interface of games is the empty interface, $\{\}$. Therefore,

$$\begin{aligned} \frac{G_1}{\text{ID}(E_2)} \circ G_2 &\equiv \frac{G_1}{\text{ID}(E_2)} \circ \frac{\text{ID}(\{\})}{G_2} \equiv \frac{G_1 \circ \text{ID}(\{\})}{\text{ID}(E_2) \circ G_2} \equiv \frac{G_1}{G_2} \\ &\equiv \frac{\text{ID}(E_1) \circ G_1}{G_2 \circ \text{ID}(\{\})} \equiv \frac{\text{ID}(E_1)}{G_2} \circ \frac{G_1}{\text{ID}(\{\})} \equiv \frac{\text{ID}(E_1)}{G_2} \circ G_1, \end{aligned}$$

which concludes the proof. $\square$

Another useful lemma is to specialize the Law of Total Probability with the uniform distribution. In Rocq, we write

```
Definition unif (n : nat) : dist nat := {code
  x ← sample uniform n ;; ret (x : nat)}.
```

to define a **dist** representing a uniform distribution of the natural numbers up to n .

Lemma 3. *For all natural numbers n and IPICK-adversaries $\mathcal{A}$, it holds that*

$$\begin{aligned} & n \cdot \Pr[\text{true} \leftarrow (\mathcal{A} \circ \text{RAND } (\text{unif } n)).\text{RUN}()] \\ &= \sum_{i < n} \Pr[\text{true} \leftarrow (\mathcal{A} \circ \text{CONST } i).\text{RUN}()]. \end{aligned}$$

Proof. In the case that $n = 0$, the equality holds trivially. It is clear that $\Pr[i \leftarrow \text{unif } n] = \frac{1}{n}$ for $i < n$. Apply Theorem 1 and multiply both sides by n . This produces the desired equation. $\square$

We turn now to the idea behind hybrid arguments. Let M be a game-pair describing the security property to be proven and G a game-pair assumed to be secure. A hybrid argument is a technique that reduces the indistinguishability of M^0 and M^1 to the indistinguishability of G^0 and G^1 . It does so by a series of $n + 1$ hybrids $(H_0, \dots, H_n)$, where the indistinguishability of each pair of consecutive hybrids reduces to the indistinguishability of G^0 and G^1 . Here M^0 is indistinguishable from $H_0 \circ G^0$ and $H_n \circ G^0$ is indistinguishable from M^1 . These assumptions are formalized as follows.

Theorem 4 (Hybrid Argument). *Let interfaces I_M and I_G be given. Furthermore, let I_M -games M^0 and M^1 and I_G -games G^0 and G^1 be given. Let a package H exporting I_M and importing $I_G \cup \text{IPICK}$ be given. Define $H_i = H \circ \frac{\text{ID}(I_G)}{\text{PICK } 1}$ and $H_\S = H \circ \frac{\text{ID}(I_G)}{\text{RAND } (\text{unif } n)}$ assuming*

$$M^0 \approx_0 H_0 \circ G^0 \quad (7)$$

$$\forall i < n, H_i \circ G^1 \approx_0 H_{i+1} \circ G^0 \quad (8)$$

$$H_n \circ G^0 \approx_0 M^1. \quad (9)$$

are satisfied, then for all I_M -adversaries $\mathcal{A}$ we have

$$\text{Adv}_M(\mathcal{A}) = n \cdot \text{Adv}_G(\mathcal{A} \circ H_\S). \quad (10)$$

Proof. By rewriting packages and applying Lemma 2, it follows that

$$H_i \circ G^b = H \circ \frac{G^b}{\text{ID}(\text{IPICK})} \circ \text{PICK } i,$$

and

$$H_{\S} \circ G^b = H \circ \frac{G^b}{\text{ID}(\text{IPICK})} \circ \text{RAND (unif } n).$$

We have

$$\begin{aligned} \text{Adv}_M(\mathcal{A}) &= |\Pr[\text{true} \leftarrow (\mathcal{A} \circ M^0).\text{RUN}()] \\ &\quad - \Pr[\text{true} \leftarrow (\mathcal{A} \circ M^1).\text{RUN}()]| \\ &= |\Pr[\text{true} \leftarrow (\mathcal{A} \circ H_0 \circ G^0).\text{RUN}()] \\ &\quad - \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_n \circ G^0).\text{RUN}()]| \\ &= |\sum_{i < n} \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_i \circ G^0).\text{RUN}()] \\ &\quad - \sum_{i < n} \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_{i+1} \circ G^0).\text{RUN}()]| \\ &= |\sum_{i < n} \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_i \circ G^0).\text{RUN}()] \\ &\quad - \sum_{i < n} \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_i \circ G^1).\text{RUN}()]| \\ &= |n \cdot \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_{\S} \circ G^0).\text{RUN}()] \\ &\quad - n \cdot \Pr[\text{true} \leftarrow (\mathcal{A} \circ H_{\S} \circ G^1).\text{RUN}()]| \\ &= n \cdot \text{Adv}_G(\mathcal{A} \circ H_{\S}) \end{aligned}$$

The second equality holds by Equations 7 and 9. The third equality holds by expanding the difference as a telescoping sum. The fourth equality holds by Equation 8. The fifth equality holds by Lemma 3. $\square$

The formal version of the hybrid argument theorem is displayed in Figure 4. For the case study, we use hybrid arguments in several incarnations. For example, when we prove many-time CPA-security from one-time CPA-security, we instantiate the game-pair M as MT_CPA and G as OT_CPA . In the other case, where we prove multi-instance many-time CPA we instantiate M by MI_MT_CPA and G by MT_CPA .

Our formulation of the hybrid argument is similar to the hybrid argument recipe given in [9, Appendix B]. However, in their work, the reduction $\mathcal{R}$ is defined informally to sample a value i and then act as $\mathcal{R}^i$. We make this argument precise and formal by the introduction of packages RAND and CONST . Finally, we also observe that inequality (7) in [9, Appendix B] can be strengthened to an equality as in Equation (10).

V. PUBLIC KEY ENCRYPTION

We turn now to the three case studies that we discuss in this paper. A public-key encryption (PKE) scheme (also called an asymmetric encryption scheme) is an encryption scheme that uses different keys for encryption and decryption. We illustrate how to carry out abstract security proofs in Nominal-SSProve using quantification over primitives.

In the first case study, we prove that, using a hybrid argument, the one-time CPA-security of *any implementation*

of the PKE scheme depicted in Figure 5 implies many-time CPA-security. The main benefit of this observation is that it lessens the burden for anyone reasoning about cryptosystems and their uses: many-time CPA is a direct consequence from one-time CPA, so it is only necessary to prove one-time CPA to obtain a many-time CPA-secure system.

In the second case study, we prove that, using a hybrid argument, the single-instance many-time CPA-security of *any implementation of the PKE scheme* implies multi-instance many-time CPA-security.

In the third case study we compose the results of case study one and two to show that the ElGamal cryptosystem is multi-instance many-time CPA-secure under the DDH computational assumption by a nested hybrid argument.

In practical terms, this means that if we are given a PKE scheme that is one-time secure, for example the ElGamal cryptosystem that we will discuss in Section V-C, we can show that we do not lose CPA-security even if we encrypt multiple times using multiple sets of keys in interleaved sessions.

Primitives are modeled in Nominal-SSProve as records, where the relevant sets defining a primitive are expressed using the built-in `choice_type`, and the cryptographic algorithms defined by the scheme are declared by their respective types. For PKE schemes, the relevant sets represent secret keys `Sec`, public keys `Pub`, messages `Mes`, and ciphertexts `Cip`. The element `sample_Cip` declares an algorithm for sampling a random ciphertext, `keygen` generates and returns a secret key and a public key that correspond to each other, `enc` encrypts a message using the public key and returns the ciphertext, and `dec` deterministically decrypts a ciphertext using the secret key and returns the message. The notation $P.(f)$ projects the field f out of record value P .

Correctness. Not all implementations of a PKE scheme are correct: At the very minimum, we should expect that the decryption of a ciphertext with the secret key that corresponds to the public key should be the identity. In state-separating proofs such a correctness property may be captured as the indistinguishability of two games CORR0 and CORR1 depicted in Figure 6. The game CORR0 implements what is commonly referred to as *real* functionality, which makes precise the steps necessary to decrypt an encrypted message m . In contrast, the game CORR1 defines the *ideal* functionality, which is, in this case, the identity. CORR0 is also referred to as *real game*, whereas CORR1 is called the *ideal game*. The definition of CORR combines CORR0 and CORR1 into a game-pair indexed by boolean value b .

Recall that adversaries are modeled by packages and that adversaries interact with games by package composition. In the general case, the game-pair $(\text{CORR0}, \text{CORR1})$ captures correctness by the following argument: If we can prove that an adversary has a low probability of distinguishing whether it has been linked (to interact with) the real or the ideal game, the PKE-scheme must have a high probability $(1 - \epsilon)$ of returning the original message after encrypting and decrypting it. We say that the PKE-scheme is ϵ -correct. In the special case of $\epsilon = 0$, we can prove that that it is impossible for the adversary

Fig. 4. The hybrid argument theorem in Rocq.

```
Theorem Adv_hybrid {IMULTI IGAME} {n : nat}
{MULTI GAME : bool → nom_package} {H A} {VA : Adversary IMULTI A}
{VG : ∀ b, Game IGAME (GAME b)} {VH : Package (unionm IGAME (IPICK nat)) IMULTI H}
: perfect IMULTI (MULTI true) (H ○ (GAME true || CONST 0))
→ perfect IMULTI (MULTI false) (H ○ (GAME true || CONST n))
→ (∀ i : nat, i < n →
  perfect IMULTI (H ○ (GAME false || CONST i)) (H ○ (GAME true || CONST i.+1)))
→ AdvOf MULTI A = AdvOf GAME (A ○ H ○ (ID IGAME || RAND (unif n))) *+ n.
```

Fig. 5. Definition of the Public-Key Encryption (PKE) primitive.

```
Record scheme :=
{ Sec : choice_type
; Pub : choice_type
; Mes : choice_type
; Cip : choice_type
; sample_Cip : dist Cip
; keygen : dist (Sec × Pub)
; enc : Pub → Mes → dist Cip
; dec : Sec → Cip → option Mes
}.
```

Fig. 6. Defining the correctness games.

```
Definition ICORR P := [interface
[ ENCDEC ] : { P.(Mes) → option P.(Mes) }
].

Definition CORR0 P :
game (ICORR P) :=
[package emptym ;
[ ENCDEC ] (m) {
' (sk, pk) ← P.(keygen) ;;
c ← P.(enc) pk m ;;
ret (P.(dec) sk c)
}
].

Definition CORR1 P :
game (ICORR P) :=
[package emptym ;
[ ENCDEC ] (m) {
ret (Some m)
}
].

Definition CORR b :=
if b then CORR0 else CORR1.
```

to distinguish if it has been linked to the real or ideal game. In this case, we say that the PKE-scheme is *perfectly correct*. In Section V-C we provide as an example an implementation of a perfectly correct PKE-scheme, the ElGamal cryptosystem, and mechanize the correctness proof in Nominal-SSProve. The definition of CPA-security does not make use of decryption, so it is only the correctness game-pair that makes a formal connection between enc and dec. Thus our formalization would be incomplete without the proof of correctness.

A. Case Study 1: Many-Time CPA-security

The theorem that we prove in Nominal-SSProve and present in this section states that the security that results from the adversary being able to challenge the ciphertext once (using one call to the encryption oracle) is enough to argue that the adversary might challenge the ciphertext arbitrarily many times (using many calls to the encryption oracle). This means to convince ourselves of the (q -times) indistinguishability under a chosen plaintext attack MT_CPA of an encryption system we have to show the one-time indistinguishability formalized as game pair OT_CPA. Fortunately, we do not have to show the property for unbounded many calls to the encryption oracle — this case is already excluded by the standard assumption that the adversary is computationally bounded.

In our formalization, we therefore count the number of calls to the encryption oracle. We have chosen to distinguish cleanly between a package that defines CPA in Figure 7 and a package that defines the counting rules in Figure 8. CPA provides two cryptographic algorithms. GEN generates and stores a public encryption key, whereas QUERY represents the call to the encryption oracle. Depending on bit b , the parameter to CPA, the oracle either returns the encryption to the plaintext message m encrypted with the public key or samples a random ciphertext from the ciphertext set. We are careful to define CPA so that it reuses the same public key for each encryption. It would be a mistake to define QUERY by calling keygen to generate a new public key on each call.

The counting package COUNT's definition is straightforward: It is parameterized by a number q that captures the maximum number of calls to an oracle. With every invocation of QUERY, an internal counter `count_loc` is incremented, if possible. Otherwise QUERY is undefined.

By package composition, we obtain the two game pairs MT_CPA and OT_CPA.

Fig. 7. Definition of ICPA and the CPA game-pair.

```

Definition ICPA P :=
[interface
  [ GEN ] : { unit ~> P.(Pub) } ;
  [ QUERY ] : { P.(Mes) ~> P.(Cip) }
].

Definition CPA P b :
game (ICPA P) :=
[package [fmap mpk_loc P] ;
  [ GEN ] (–) {
    '(_, pk) ← P.(keygen) ;;
    #put mpk_loc P := Some pk ;;
    ret pk
  } ;
  [ QUERY ] (m) {
    pk ← getSome mpk_loc P ;;
    if b then
      P.(enc) pk m
    else
      P.(sample_Cip)
  }
].

```

Fig. 8. Definition of a counting package for ICPA.

```

Definition COUNT P q :
package (ICPA P) (ICPA P) :=
[package [fmap count_loc] ;
  [ GEN ] (–) {
    call [ GEN ] tt
  } ;
  [ QUERY ] (m) {
    count ← get count_loc ;;
    #assert count < q ;;
    #put count_loc := count.+1 ;;
    call [ QUERY ] m
  }
].

```

Fig. 9. Definition of the SLIDE hybrid.

```

Definition SLIDE P q : package
  (unionm (ICPA P) (IPICK nat)) (ICPA P) :=
[package [fmap count_loc' ; mpk_loc' P] ;
  [ GEN ] (–) {
    pk ← call [ GEN ] tt ;;
    #put mpk_loc' P := Some pk ;;
    ret pk
  } ;
  [ QUERY ] (m) {
    count ← get count_loc' ;;
    #assert (count < q) ;;
    #put count_loc' := count.+1 ;;
    i ← call [ PICK ] tt ;;
    pk ← getSome mpk_loc' P ;;
    if (count < i) then
      c ← P.(sample_Cip) ;; ret c
    else if (i < count) then
      c ← P.(enc) pk m ;; ret c
    else
      call [ QUERY ] m
  }
].

```

```

Notation MT_CPA P q :=
  (λ b, COUNT P q ◦ CPA P b).
Notation OT_CPA P :=
  (λ b, COUNT P 1 ◦ CPA P b).

```

In summary, MT_CPA and OT_CPA define game-pairs in the real vs. ideal paradigm. For q , $MT_CPA\ q\ true$ and $OT_CPA\ true$ define the real functionality, respectively, by once encrypting a message that is chosen by the adversary and allowing the adversary many or just one call to the encryption oracle. $MT_CPA\ q\ false$ and $OT_CPA\ false$ define the ideal functionality by sampling a random ciphertext. The entire security argument is built on the indistinguishability between the respective games.

Below, we prove formally Theorem $Adv_MT_CPA_OT$, which states that MT_CPA reduces to OT_CPA . At first glance, this is a surprising result, because it is not at all obvious that the scheme guarantees indistinguishability under chosen plaintext attack while we reuse the same public key multiple times to produce different ciphertexts without revealing some common pattern in the outputs that may help the adversary to gain some kind of advantage. On the contrary, once the adversary is in possession of the public key, the adversary is free to encrypt as many messages as desired — the scheme is secure. In comparison, a similar argument cannot be made in the setting of symmetric encryptions: Using the one-time pad, it is not possible to reduce indistinguishability of multiple encryptions to the indistinguishability of one encryption because the key used to encrypt is not revealed.

The proof of Theorem $Adv_MT_CPA_OT$ proceeds by a hybrid argument. Following the outline of our hybrid arguments above, we first define the hybrid called $SLIDE$. The first parameter refers to an abstract implementation of the PKE primitive P , which allows $SLIDE$ to access the relevant cryptographic algorithms provided by the implementation. The second parameter, q , is an upper bound for the number of the number of calls the adversary may initiate to the (encryption) oracle. Furthermore, $SLIDE$ obtains the parameter i by calling $PICK$. In the current formalization, depicted in Figure 9, hybrid arguments cannot be parameterized by oracle names. Hence, the call to $QUERY$ is baked into $SLIDE$.

$SLIDE$ is designed to play the role of H from the introduction. Let us consider the two corner cases first. If $i = 0$ then all messages sent from the adversary to $SLIDE$ are responded to as if they originate from the left game and if $i = n$ then all messages are responded to as if they originate from the right game. In all other cases, $0 \leq i \leq q$, $SLIDE$ responds to the first i calls simulating the right game and all others simulating the left game, gradually shifting the focus (in n -steps) from the left game to right. This functionality is implemented in $SLIDE$ using an internal counter $count_loc'$ to distinguish it from the internal counter $count_loc$ from the $COUNT$ package (see Figure 8). The counter $count_loc'$ counts the number of calls to the encryption oracle. When the counter is less than i , then a ciphertext is sampled, which corresponds to calling the

encryption oracle using the right game. When the counter is exactly i , then SLIDE calls QUERY, i.e., the encryption oracle provided by the underlying package. When the counter is greater than i , the encryption oracle encrypts the message per the left game.

We prove two lemmas about SLIDE. The first lemma shows that SLIDE behaves just as MT_CPA^0 when $i = 0$ and as MT_CPA^1 when $i = n$. This lemma establishes two of the desired properties for SLIDE being used as H in the hybrid arguments and covers therefore assumptions (7) and (9) for Theorem 4 to be applicable.

Lemma $MT_CPA_SLIDE_perfect \{P\ q\} \ b :$
 $perfect \ (ICPA\ P) \ (MT_CPA\ P\ q\ b) \ (SLIDE\ P\ q$
 $\circ \ (OT_CPA\ P\ true \ ||$
 $\quad CONST \ (if\ b\ then\ 0\ else\ q)))$.

The second lemma shows that when the underlying package encrypts the right message we might as well increase the parameter to $i + 1$. This satisfies assumption (8). The proof is completed by considering the different cases for the relative size of the SLIDE counter and i .

Lemma $SLIDE_succ_perfect \{P\ q\ i\} :$
 $perfect \ (ICPA\ P)$
 $\quad (SLIDE\ P\ q \circ$
 $\quad \quad (OT_CPA\ P\ false \ || \ CONST\ i \ \ \))$
 $\quad (SLIDE\ P\ q \circ$
 $\quad \quad (OT_CPA\ P\ true \ \ \ || \ CONST\ i.+1)))$.

We define an abbreviation for the reduction.

Notation $OTM\ P\ q := (SLIDE\ P\ q \circ$
 $\quad (ID\ (ICPA\ P) \ || \ RAND\ (unif\ q)))$.

And combine the lemmas by hybrid argument Adv_hybrid (Theorem 4) to prove the following reduction from MT_CPA to OT_CPA .

Theorem $Adv_MT_CPA_OT\ P\ q\ A$
 $\quad \{Adversary\ (ICPA\ P)\ A\} :$
 $AdvOf\ (MT_CPA\ P\ q)\ A =$
 $\quad AdvOf\ (OT_CPA\ P)\ (A \circ OTM\ P\ q) \ **\ q$.

B. Case Study 2: Multi-Instance CPA-security

In this case study, we prove by hybrid argument that multi-instance CPA-security depicted in Figure 10 reduces to single-instance CPA-security depicted in Figure 7. The multi-instance interface and game-pair are parameterized by n , which defines the number of instances. The multi-instance interface is different from the single-instance interface, in that each query takes an extra parameter $'fin'$ in specifying which instance to interact with. Each instance has its own public key, which is saved in the `pks_loc` location.

Figure 11 defines a multi-instance counting package. Besides n , the package is additionally parameterized by q denoting the maximal number of queries allowed per instance. This means that there are n counters, one counter for each instance.

We capture multi-instance many-time CPA-security by the following game.

Notation $MI_MT_CPA\ P\ n\ q :=$
 $\quad (\lambda\ b, \ MI_COUNT\ P\ n\ q \circ MI_CPA\ P\ n\ b)$.

The next step is then to define a hybrid package HYB_MI_CPA , depicted in Figure 12, to reduce *multi*-instance many-time CPA-security to *single*-instance many-time CPA-security. In the figure j refers to a specific instance of many-time CPA-security and i determines which hybrid is considered. In the case that $j > i$, the hybrid implements the real behavior of CPA-security and in the case that $j < i$, the hybrid implements the ideal behavior of CPA-security. In the case that $j == i$, calls are forwarded to the underlying (single-instance) package.

First, we show that HYB_MI_CPA correctly represents the behaviors of the real and ideal MI_MT_CPA for the two edge cases $CONST\ 0$ to satisfy assumption (7), and $CONST\ n$ to satisfy assumption (9). Note the variable b , selecting either case.

Lemma $MI_CPA_0_n\ P\ n\ q\ b :$
 $perfect \ (IMI_CPA\ P\ n)$
 $\quad (HYB_MI_CPA\ P\ n\ q \circ (MT_CPA\ P\ q\ true \ ||$
 $\quad \quad CONST \ (if\ b\ then\ 0\ else\ n)))$.

Next we show that we may step from one hybrid i to the next hybrid $i+1$ to satisfy assumption 8 by switching from the ideal game (selected by argument `false`) to the real game (selected by argument `true`).

Lemma $MI_CPA_i\ P\ n\ q\ i :$
 $perfect \ (IMI_CPA\ P\ n)$
 $\quad (HYB_MI_CPA\ P\ n\ q \circ$
 $\quad \quad (MT_CPA\ P\ q\ false \ || \ CONST\ i \ \ \))$
 $\quad (HYB_MI_CPA\ P\ n\ q \circ$
 $\quad \quad (MT_CPA\ P\ q\ true \ \ \ || \ CONST\ i.+1)))$.

We define an abbreviation for the multi-instance hybrid reduction.

Notation $STM\ P\ n\ q := (HYB_MI_CPA\ P\ n\ q \circ$
 $\quad (ID\ (ICPA\ P) \ || \ RAND\ (unif\ n)))$.

As promised, the final theorem of the case study states that *multi*-instance many-time CPA-security reduces to *single*-instance many-time CPA-security.

Theorem $Adv_MI_CPA_SI\ P\ n\ q\ A$
 $\quad \{Adversary\ (IMI_CPA\ P\ n)\ A\} :$
 $AdvOf\ (MI_MT_CPA\ P\ n\ q)\ A =$
 $\quad AdvOf\ (MT_CPA\ P\ q)\ (A \circ STM\ P\ n\ q) \ **\ n$.

We prove the final theorem by application of Theorem 4. $MT_CPA_0_n$ proves the base cases and MT_CPA_i proves the iteration case.

C. Case Study 3: Multi-Instance Many-Time CPA-security for ElGamal

In this case study, we show multi-instance many-time CPA-security for ElGamal by simple specialization and composition

Fig. 10. Definition of Multi-Instance CPA.

```

Definition IMI_CPA P n := [interface
  [ GEN ] : { 'fin n ~> P.(Pub) } ;
  [ QUERY ] : { 'fin n × P.(Mes) ~> P.(Cip) }
].

Definition MI_CPA P n b :
  game (IMI_CPA P n) :=
  [package [fmap pks_loc P] ;
  [ GEN ] (j) {
    '(_, pk) ← P.(keygen) ;;
    #put pks_loc P [ j ] := pk ;;
    ret pk
  } ;
  [ QUERY ] '(j, m) {
    pk ← get pks_loc P [ j ] ;;
    if b then
      P.(enc) pk m
    else
      P.(sample_Cip)
  }
].

```

Fig. 11. Counting package for Multi-Instance CPA.

```

Definition MI_COUNT P n q :
  package (IMI_CPA P n) (IMI_CPA P n) :=
  [package [fmap counts_loc] ;
  [ GEN ] (j) {
    call [ GEN ] j
  } ;
  [ QUERY ] '(j, m) {
    counts ← get counts_loc ;;
    let countj := odflt 0 (counts j) in
    #assert countj < q ;;
    c ← call [ QUERY ] (j, m) ;;
    #put counts_loc :=
      setm counts j countj.+1 ;;
    ret c
  }
].

```

Fig. 13. ElGamal definition

```

Definition elgamal : scheme :=
  { | Sec := 'exp G
    ; Pub := 'el G
    ; Mes := 'el G
    ; Cip := 'el G × 'el G
    ; sample_Cip := {code
      c1 ← sample uniform #|el G| ;;
      c2 ← sample uniform #|el G| ;;
      ret (c1, c2) }
    ; keygen := {code
      sk ← sample uniform #|exp G| ;;
      ret (sk, 'g ^ sk) }
    ; enc := λ pk m, {code
      r ← sample uniform #|exp G| ;;
      ret ('g ^ r, m * (pk ^ r)) }
    ; dec := λ sk '(c1, c2),
      Some (c2 * (c1 ^- sk))
  }.

```

Fig. 12. Hybrid package for reducing Multi-Instance CPA-security to Single-Instance CPA.

```

Definition HYB_MI_CPA P n q :
  package (unionm (ICPA P) (IPICK nat))
  (IMI_CPA P n) :=
  [package [fmap pks_loc P ; counts_loc] ;
  [ GEN ] (j) {
    i ← call [ PICK ] tt ;;
    pk ← if i == j then
      call [GEN] tt
    else
      '(_, pk) ← P.(keygen) ;;
      ret pk ) ;;
    #put pks_loc P [ j ] := pk ;;
    ret pk
  } ;
  [ QUERY ] '(j, m) {
    counts ← get counts_loc ;;
    let countj := odflt 0 (counts j) in
    #assert countj < q ;;
    pk ← get pks_loc P [ j ] ;;
    i ← call [ PICK ] tt ;;
    c ← ( if j < i then
      c ← P.(sample_Cip) ;; ret c
    else if i < j then
      c ← P.(enc) pk m ;; ret c
    else
      call [QUERY] m ) ;;
    #put counts_loc :=
      setm counts j countj.+1 ;;
    ret c
  }
].

```

Fig. 14. DDH Definition

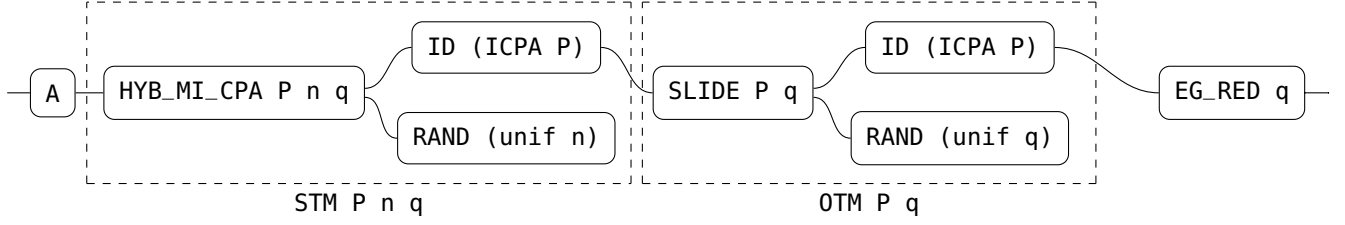
```

Definition IDDH G :=
  [interface [ GETABC ] :
    { 'unit ~> 'el G × 'el G × 'el G } ].

Definition DDH G bit :
  game (IDDH G) :=
  [package emptym ;
  [ GETABC ] 'tt {
    a ← sample uniform #|exp G| ;;
    b ← sample uniform #|exp G| ;;
    if bit then
      ret ('g ^ a, 'g ^ b, ('g ^ a) ^ b)
    else
      c ← sample uniform #|exp G| ;;
      ret ('g ^ a, 'g ^ b, 'g ^ c)
  }
].

```

Fig. 15. The full adversary of the final theorem `MI_MT_CPA_elgamal` with $P = \text{elgamal}$. `STM` defines the reduction from single-instance to multi-instance and `OTM` defines the reduction from many-time to one-time CPA.



of existing lemmas. First, we introduce an implementation of our PKE primitive, namely the ElGamal Public Key Scheme.

The ElGamal Public Key Scheme. We introduce an implementation of the ElGamal cryptosystem as a public key scheme seen in Figure 13. The implementation depends on a cyclic group G that has ' g ' as a generator. ' $el \in G$ ' and ' $exp \in G$ ' represents the elements and exponents of G respectively. The operators $*$, $^$, $^-$ represent group multiplication, exponentiation and inverse exponentiation respectively. The fact that Nominal-SSProve is a shallow embedding allows us to inherit the group structures and theorems of `mathcomp` [16] now based on Hierarchy Builder [11].

ElGamal is perfectly correct. We prove that ElGamal is perfectly correct, i.e. for all adversaries (even the computationally unconstrained) the two correctness games `CORR0` and `CORR1` are perfectly indistinguishable. In essence, the proof is about solving the usual group equations.

```
Theorem correct_elgamal :
  perfect (I.CORR elgamal)
    (CORR0 elgamal) (CORR1 elgamal).
```

OT_CPA for ElGamal reduces to DDH. We prove that we can find a reduction such that for any adversary, the chance that the adversary distinguishes between `OT_CPA` games is the same as the adversary distinguishing between the DDH games through the reduction. Therefore, by contra-position: If we believe that the DDH assumption is strong enough for the group in question, we must believe in the `OT_CPA`-security for ElGamal.

We define the DDH games in Figure 14 and show the following result, which relies on a `EG_RED` that we have omitted in the interest of space.

```
Lemma OT_CPA_elgamal {A}
  {Adversary (ICPA elgamal) A} :
  AdvOf (OT_CPA elgamal) A =
    AdvOf (DDH G) (A o EG_RED).
```

This is the usual security result presented for ElGamal such as in [4], [3], [20], [17]. We shall now see that in our formulation, it can be iterated in two dimensions.

MT_CPA for ElGamal reduces to DDH. To show the application of Case Study 1 we formally prove, that many-time

CPA-security reduces to DDH for ElGamal. We prove the theorem by combining the `MT_CPA` to `OT_CPA` reduction and the ElGamal-specific `OT_CPA` to DDH reduction.

```
Lemma MT_CPA_elgamal q {A}
  {Adversary (ICPA elgamal) A} :
  AdvOf (MT_CPA elgamal q) A =
    AdvOf (DDH G)
      ((A o OTM elgamal q) o EG_RED) ** q.
```

MI_MT_CPA for ElGamal reduces to DDH. Finally, we combine the results of Case Study 2 with the previous lemma to show that multi-instance many-time CPA-security for ElGamal reduces to the DDH computational assumption.

```
Theorem MI_MT_CPA_elgamal n q {A}
  {Adversary (IMI_CPA elgamal n) A} :
  AdvOf (MI_MT_CPA elgamal n q) A =
    AdvOf (DDH G) (((A o STM elgamal n q)
      o OTM elgamal q) o EG_RED) ** q ** n.
```

Figure 15 shows the adversary of the right-hand side of the equation. The `RAND` game appears twice, but this is not an issue. The package composition semantics based nominal sets ensure that each `RAND` package has its own cell location, so that each `RAND` package picks a random and independent value to return on calls to `PICK`.

VI. CONCLUSION

In this paper we develop a theory for nested hybrid arguments in state-separating proofs that encompasses multi-instance and query-counting security proofs. We mechanize the technique in Nominal-SSProve. Our work is to our knowledge the first mechanization of hybrid arguments in this generality that can be arbitrarily nested. We demonstrate the versatility of our approach in the three case studies about public-key encryption.

REFERENCES

- [1] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, and Vitor Pereira. A fast and verified software stack for secure function evaluation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, page 1989–2006, New York, NY, USA, 2017. Association for Computing Machinery.
- [2] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In *Annual International Cryptology Conference*, pages 358–389. Springer, 2023.

- [3] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *Proceedings of the 31st Annual Conference on Advances in Cryptology, CRYPTO'11*, page 71–90, Berlin, Heidelberg, 2011. Springer-Verlag.
- [4] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Formal certification of elgamal encryption. In Paolo Degano, Joshua Guttman, and Fabio Martinelli, editors, *Formal Aspects in Security and Trust*, pages 1–19, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [5] Mihir Bellare, Alexandra Boldyreva, and Silvio Micali. Public-key encryption in a multi-user setting: security proofs and improvements. In *Proceedings of the 19th International Conference on Theory and Application of Cryptographic Techniques, EUROCRYPT'00*, page 259–274, Berlin, Heidelberg, 2000. Springer-Verlag.
- [6] Mihir Bellare, Anand Desai, David Pointcheval, and Phillip Rogaway. Relations among notions of security for public-key encryption schemes. In Hugo Krawczyk, editor, *Advances in Cryptology — CRYPTO '98*, pages 26–45, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
- [7] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006*, pages 409–426, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [8] Bruno Blanchet, Pierre Boutry, Christian Doczkal, Benjamin Grégoire, and Pierre-Yves Strub. Cv2ec: Getting the best of both worlds. In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, pages 279–294, 2024.
- [9] Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. State separation for code-based game-playing proofs. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 222–249, Cham, 2018. Springer International Publishing.
- [10] Ran Canetti, Alley Stoughton, and Mayank Varia. Easyuc: Using easycrypt to mechanize proofs of universally composable security. In *32nd IEEE Computer Security Foundations Symposium, CSF 2019, Hoboken, NJ, USA, June 25-28, 2019*, pages 167–183. IEEE, 2019.
- [11] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In Zena M. Ariola, editor, *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020)*, volume 167 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 34:1–34:21, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [12] François Dupressoir, Andreas Hülsing, Cameron Low, Matthias Meijers, Charlotte Mylog, and Sabine Oechsner. *How Hard can it be to Formalize a Proof?: Lessons from Formalizing CryptoBox Three Times in EasyCrypt*, page 137–167. Springer Nature Singapore, December 2025.
- [13] François Dupressoir, Konrad Kohbrok, and Sabine Oechsner. Bringing state-separating proofs to easycrypt a security proof for cryptobox. In *2022 IEEE 35th Computer Security Foundations Symposium (CSF)*, pages 227–242, 2022.
- [14] Ross Evans, Matthew McKague, and Douglas Stebila. ProofFrog: A tool for verifying game-hopping proofs. *Cryptology ePrint Archive*, Paper 2025/418, 2025.
- [15] Marc Fischlin and Arno Mittelbach. An overview of the hybrid argument. *Cryptology ePrint Archive*, Paper 2021/088, 2021.
- [16] Georges Gonthier, Assia Mahboubi, Laurence Rideau, Enrico Tassi, and Laurent Théry. A modular formalisation of finite group theory. In Klaus Schneider and Jens Brandt, editors, *Theorem Proving in Higher Order Logics*, pages 86–101, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [17] Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Carmine Abate, Nikolaj Sidorenko, Cătălin Hrițcu, Kenji Maillard, and Bas Spitters. Ssprove: A foundational framework for modular cryptographic proofs in coq. *ACM transactions on programming languages and systems*, 45(3):1–61, 2023.
- [18] Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. Formal verification of saber’s public-key encryption scheme in easycrypt. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 622–653, Cham, 2022. Springer Nature Switzerland.
- [19] Markus Krabbe Larsen and Carsten Schürmann. Nominal state-separating proofs. In *2025 IEEE 38th Computer Security Foundations Symposium (CSF)*, pages 363–377, 2025.
- [20] Adam Petcher and Greg Morrisett. The foundational cryptography framework. In Riccardo Focardi and Andrew Myers, editors, *Principles of Security and Trust*, pages 53–72, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [21] Mike Rosulek. *The Joy of cryptography: An undergraduate course in provable security*. MIT Press, London, England, January 2026.
- [22] T.Y.C. Woo and S.S. Lam. ‘authentication’ revisited (correction and addendum to ‘authentication’ for distributed systems, jan. 92, 39-52). *Computer*, 25(3):10–, 1992.
- [23] Andrew C. Yao. Theory and application of trapdoor functions. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 80–91, 1982.

APPENDIX

A. Artifacts

All results of this paper have been mechanized in Nominal-SSProve, and the source files are attached to this submission as an artifact⁴. The code is designed as a pull-request for SSProve, so a full copy of the repository is included. See `diff.txt` for a complete list of changes. The source files in directory `theories/Crypt` relevant for this paper are:

- `../nominal/packages/TotalProbability.v`
Defines and proves the law of total probability (Section III).
- `../nominal/packages/HybridArgument.v`
Defines and proves the hybrid argument (Section IV).
- `../examples/PKE/Scheme.v`
Models PKE and CPA-security (Section V).
- `../examples/PKE/OneToMany.v`
Models case study 1 (Section V-A).
- `../examples/PKE/MultiInstance.v`
Models case study 2 (Section V-B).
- `../examples/PKE/LDDH.v`
Models the DDH assumption used in case study 3 (Section V-C).
- `../examples/PKE/ElGamal.v`
Models case study 3 (Section V-C).

We provide three different ways to install the required dependencies. Follow one of them in the directory of the repository.

- **opam:** Run the following commands

```
opam repo add coq-released \
https://coq.inria.fr/opam/released
opam update
opam install . --deps-only
```

to install the required dependencies.

- **nix:** Run `nix-shell` to enter the development environment.
- **docker:** Run the following commands

```
docker build -t csf26-review .
docker run --rm -it csf26-review .
```

to build the image and to enter the development environment.

The files are checked using the command `make` while inside one of the three development environments.

⁴<https://github.com/MarkusKL/ssprove/tree/for-submission-jan26>